
TP4 BDA: Introduction au Procedural Language (PL/SQL) d'Oracle

ING2 INFORMATIQUE
Sup Galilée

Réalisé par
Daniel THARMARAJAH 12000230

Contents

Exercice 1	3
1.	3
2.	4
3.	4
4.	4
Création de la table <code>resultatFactoriel</code>	4
5.	5
Exercice n°2	5
Création de la table <code>emp</code>	5
Insertion des données initiales	5
1.	5
2.	6
3.	6
4.	6
5.	7
6.	7
Exercice 3	8
Création de la table <code>client</code>	8
Implémentation du package	8

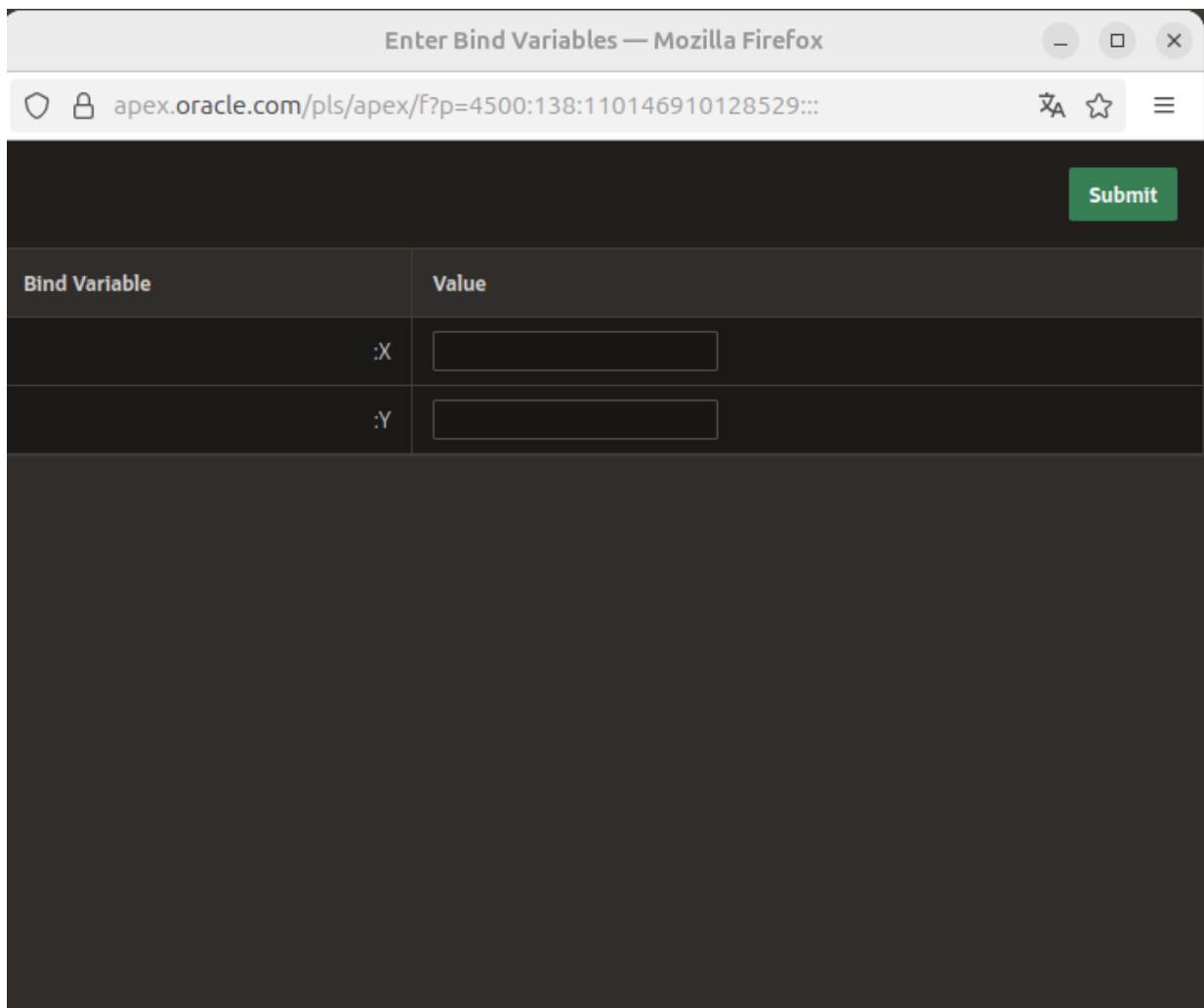
Tout le TP a été réalisé sur APEX Oracle

Exercice 1

1.

```
DECLARE
v_1 NUMBER := :x;
v_2 NUMBER := :y;
BEGIN
    dbms_output.put_line('-----');
    dbms_output.put_line('La somme des deux nombres introduits est ' || (v_1+v_2));
END;
```

Ce code ouvre une fenêtre dans laquelle on peut renseigner des valeurs, grâce à l'utilisation de :x et :y.



The screenshot shows a web browser window titled "Enter Bind Variables — Mozilla Firefox". The address bar displays the URL "apex.oracle.com/pls/apex/f?p=4500:138:110146910128529:::". The main content area is a dark-themed form with a table for bind variables. The table has two columns: "Bind Variable" and "Value". There are two rows: one for ":X" and one for ":Y", each with an adjacent text input field. A green "Submit" button is located in the top right corner of the form area.

Bind Variable	Value
:X	
:Y	

2.

```
DECLARE
v_1 NUMBER := :x;
BEGIN
    dbms_output.put_line('Voici sa table de multiplication');
    FOR i IN 1..10 LOOP
        dbms_output.put_line(v_1 * i);
    END LOOP;
END;
```

3.

```
CREATE OR REPLACE FUNCTION puissance(x NUMBER, n NUMBER)
RETURN NUMBER IS
BEGIN
    IF n = 0 THEN
        RETURN 1;
    ELSE
        RETURN x * puissance(x, n - 1);
    END IF;
END;
```

4.

Création de la table resultatFactoriel

```
CREATE TABLE resultatFactoriel
( entier number(10) NOT NULL,
  resultat number(10) NOT NULL,
  PRIMARY KEY (entier,resultat)
);
```

```
DECLARE
v_res resultatFactoriel%ROWTYPE;
v_n NUMBER;
BEGIN
    v_n := :n;
    v_res.entier := v_n;
    v_res.resultat := 1;
    FOR i in 1..v_n LOOP
        v_res.resultat := v_res.resultat * i;
    END LOOP;
    INSERT into resultatFactoriel VALUES v_res ;
END;
```

5.

```
DECLARE
v_res resultatFactoriel%ROWTYPE;
BEGIN
FOR i in 1..20 LOOP
    v_res.entier := i;
    v_res.resultat := 1;
    FOR j IN 1..i LOOP
        v_res.resultat := v_res.resultat * j;
    END LOOP;
    INSERT into resultatFactoriel VALUES v_res ;
END LOOP;
END;
```

Exercice n°2

Création de la table emp

```
CREATE TABLE emp
( matr NUMBER (10) NOT NULL ,
  nom VARCHAR2 (50) NOT NULL ,
  sal number (7 ,2) ,
  adresse VARCHAR2 (96) ,
  dep NUMBER (10) NOT NULL ,
  CONSTRAINT emp_pk PRIMARY KEY (matr)
);
```

Insertion des données initiales

```
INSERT INTO emp (matr, nom, sal, adresse, dep) VALUES
(0, 'Armand', 1000, 'Saint-Didier-des-Bois', 93430);
INSERT INTO emp (matr, nom, sal, adresse, dep) VALUES
(1, 'Hebert', 3600, 'Marigny-le-Châtel', 93800);
INSERT INTO emp (matr, nom, sal, adresse, dep) VALUES (2, 'Ribeiro', 9000, 'Maillères', 13003);
INSERT INTO emp (matr, nom, sal, adresse, dep) VALUES (3, 'Savary', 1200, 'Conie-Molitard', 10);
```

1.

```
DECLARE
v_employe emp%ROWTYPE;
BEGIN
    v_employe.matr := 4;
    v_employe.nom := 'Youcef';
    v_employe.sal := 2500;
    v_employe.adresse := 'avenue Anatole France';
    v_employe.dep := 92000;
    INSERT INTO emp VALUES v_employe;
END;
```

2.

```
DECLARE
v_nb_lignes NUMBER;
BEGIN
    DELETE FROM emp WHERE dep = 10;
    v_nb_lignes := SQL%ROWCOUNT;
    dbms_output.put_line('v_nb_lignes : ' || v_nb_lignes);
END;
```

3.

```
DECLARE
CURSOR c_listeSalaires IS SELECT sal FROM emp;
v_salaire emp.sal%TYPE;
v_somme emp.sal%TYPE := 0;

BEGIN
    OPEN c_listeSalaires;
    LOOP
        FETCH c_listeSalaires INTO v_salaire;
        EXIT WHEN c_listeSalaires%NOTFOUND;
        IF v_salaire IS NOT NULL THEN
            v_somme := v_somme + v_salaire;
        END IF;
    END LOOP;
    CLOSE c_listeSalaires;
    dbms_output.put_line('total : ' || v_somme);
END;
```

4.

```
DECLARE
CURSOR c_listeSalaires IS SELECT sal FROM emp;
v_salaire emp.sal%TYPE;
v_somme emp.sal%TYPE := 0;
v_nb NUMBER := 0;

BEGIN
    OPEN c_listeSalaires;
    LOOP
        FETCH c_listeSalaires INTO v_salaire;
        EXIT WHEN c_listeSalaires%NOTFOUND;
        IF v_salaire IS NOT NULL THEN
            v_somme := v_somme + v_salaire;
        END IF;
    END LOOP;
    v_nb := c_listeSalaires%ROWCOUNT;
    CLOSE c_listeSalaires;
    dbms_output.put_line('avg : ' || v_somme / v_nb);
END;
```

5.

```
DECLARE
CURSOR c_listeSalaires IS SELECT sal FROM emp;
v_somme emp.sal%TYPE := 0;

BEGIN
  FOR v_emp IN c_listeSalaires LOOP
    v_somme := v_somme + v_emp.sal;
  END LOOP;
  dbms_output.put_line('total : ' || v_somme);
END;
```

```
DECLARE
CURSOR c_listeSalaires IS SELECT sal FROM emp;
v_somme emp.sal%TYPE := 0;
v_nb NUMBER := 0;

BEGIN
  FOR v_emp IN c_listeSalaires LOOP
    v_nb := c_listeSalaires%ROWCOUNT;
    v_somme := v_somme + v_emp.sal;
  END LOOP;
  dbms_output.put_line('total : ' || v_somme / v_nb);
END;
```

6.

```
DECLARE
CURSOR c(p_dep EMP.dep%TYPE) IS
  SELECT dep, nom FROM emp WHERE dep = p_dep;
BEGIN
  FOR v_employe IN c(92000) LOOP
    dbms_output.put_line('Dep 1 : ' || v_employe.nom);
  END LOOP;

  FOR v_employe IN c(75000) LOOP
    dbms_output.put_line('Dep 2 : ' || v_employe.nom);
  END LOOP;
END;
```

Exercice 3

Création de la table client

```
CREATE TABLE client
( idClient NUMBER(10) NOT NULL,
  nomClient VARCHAR2(50) NOT NULL,
  prenomClient VARCHAR2(50) NOT NULL,
  codePostalClient VARCHAR2(96),
  ageClient NUMBER(4),
  CONSTRAINT client_pk PRIMARY KEY (idClient)
);
```

Implémentation du package

```
CREATE OR REPLACE PACKAGE BODY packageClient IS

  -- Procédure d'ajout d'un client
  procedure addClient(p_idClient client.idClient%TYPE, p_nomClient client.nomClient%TYPE,
    p_prenomClient client.prenomClient%TYPE, p_codePostalClient client.codePostalClient%TYPE,
    p_ageClient client.ageClient%TYPE) IS
    v_client client%ROWTYPE;
  BEGIN
    v_client.idClient := p_idClient;
    v_client.nomClient := p_nomClient;
    v_client.prenomClient := p_prenomClient;
    v_client.codePostalClient := p_codePostalClient;
    v_client.ageClient := p_ageClient;
    INSERT INTO client VALUES v_client;
  END;

  -- Surchargée : Procédure d'ajout d'un client sans âge
  procedure addClient(p_idClient client.idClient%TYPE, p_nomClient client.nomClient%TYPE,
    p_prenomClient client.prenomClient%TYPE, p_codePostalClient client.codePostalClient%TYPE) IS
  BEGIN
    v_client client%ROWTYPE;
    v_client.idClient := p_idClient;
    v_client.nomClient := p_nomClient;
    v_client.prenomClient := p_prenomClient;
    v_client.codePostalClient := p_codePostalClient;
    INSERT INTO client VALUES v_client;
  END;

  -- Fonction pour calculer l'âge moyen des clients
  function getAgeAvg return number IS
    v_ageMoyen client.ageClient%TYPE;
  BEGIN
    SELECT avg(ageClient) INTO v_ageMoyen FROM client;
    RETURN v_ageMoyen;
  END;

END; /
```